



# Tracing and profiling



# strace

System call tracer - https://strace.io

- Available on all GNU/Linux systems Can be built by your cross-compiling toolchain generator or by your build system.
- Allows to see what any of your processes is doing: accessing files, allocating memory... Often sufficient to find simple bugs.
- Usage:
  `strace <command>` (starting a new process)
  `strace -f <command>` (**f**ollow child processes too)
  `strace -p <pid>` (tracing an existing process)
  `strace -c <command>` (time statistics per system call)
  `strace -e <expr> <command>` (use **e**xpression for advanced filtering)

See the strace manual for details



Image credits: https://strace.io/



# strace example output

```
> strace cat Makefile
[...]
fstat64(3, {st_mode=S_IFREG|0644, st_size=111585, ...}) = 0
mmap2(NULL, 111585, PROT_READ, MAP_PRIVATE, 3, 0) = 0xb7f69000
close(3) = 0
access("/etc/ld.so.nohwcap", F_OK) = -1 ENOENT (No such file or directory)
open("/lib/tls/i686/cmov/libc.so.6", O_RDONLY) = 3
read(3, "\177ELF\1\1\1\0\0\0\0\0\0\0\0\0\3\0\3\0\1\0\0\0320h\1\0004\0\0\0344"..., 512) = 512
fstat64(3, {st_mode=S_IFREG|0755, st_size=1442180, ...}) = 0
mmap2(NULL, 1451632, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0xb7e06000
mprotect(0xb7f62000, 4096, PROT_NONE) = 0
mmap2(0xb7f66000, 9840, PROT_READ|PROT_WRITE,
    MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xb7f66000
close(3) = 0
[...]
openat(AT_FDCWD, "Makefile", O_RDONLY)  = 3
newfstatat(3, "", {st_mode=S_IFREG|0644, st_size=173, ...}, AT_EMPTY_PATH) = 0
fadvise64(3, 0, 0, POSIX_FADV_SEQUENTIAL) = 0
mmap(NULL, 139264, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0x7f7290d28000
read(3, "ifneq ($(KERNELRELEASE),)\nobj-m "..., 131072) = 173
write(1, "ifneq ($(KERNELRELEASE),)\nobj-m "..., 173ifneq ($(KERNELRELEASE),)
```

Hint: follow the open file descriptors returned by `open()`. This tells you what files are handled by further system calls.



# strace filtering

- Display only a specific set of system calls:

```
$ strace -e 'openat,write' cat Makefile
```

- Filter out specific system calls:

```
$ strace -e '!poll' cat Makefile
```

- Show only system calls returning a specific status

```
$ strace -e 'status=failed' cat Makefile
```

- Trace how a file is accessed and used among different system calls

```
$ strace -P '/etc/ld.so.cache' cat Makefile
```

- Run `strace --tips` to learn new commands !



# ltrace

A tool to trace **shared** library calls used by a program and all the signals it receives

- Very useful complement to `strace`, which shows only system calls.
- Of course, works even if you don't have the sources
- Allows to filter library calls with regular expressions, or just by a list of function names.
- With the `-S` option it shows system calls too!
- Also offers a summary with its `-c` option.
- Manual page: https://linux.die.net/man/1/ltrace
- Works better with *glibc*. `ltrace` used to be broken with *uClibc* (now fixed), and is not supported with *Musl* (Buildroot 2022.11 status).

See https://en.wikipedia.org/wiki/Ltrace for details



# ltrace example output

```
# ltrace ffmpeg -f video4linux2 -video_size 544x288 -input_format mjpeg -i /dev/video0 \
     -pix_fmt rgb565le -f fbdev /dev/fb0
__libc_start_main([ "ffmpeg", "-f", "video4linux2", "-video_size"... ] <unfinished ...>
setvbuf(0xb6a0ec80, nil, 2, 0)                    = 0
av_log_set_flags(1, 0, 1, 0)                      = 1
strchr("f", ':')                                  = nil strlen("f")                                  =
1
strncmp("f", "L", 1)                              = 26
strncmp("f", "h", 1)                              = -2
strncmp("f", "?", 1)                              = 39
strncmp("f", "help", 1)                           = -2
strncmp("f", "-help", 1)                          = 57
strncmp("f", "version", 1)                        = -16
strncmp("f", "buildconf", 1)                      = 4
strncmp("f", "formats", 1)                        = 0
strlen("formats")                                 = 7
strncmp("f", "muxers", 1)                         = -7
strncmp("f", "demuxers", 1)                       = 2
strncmp("f", "devices", 1)                        = 2
strncmp("f", "codecs", 1)                         = 3
...
```



# ltrace summary

Example summary at the end of the ltrace output (`-c` option)

```
% time     seconds  usecs/call     calls      function
------ ----------- ----------- --------- --------------------
 52.64    5.958660     5958660         1 __libc_start_main
 20.64    2.336331     2336331         1 avformat_find_stream_info
 14.87    1.682895         421      3995 strncmp
  7.17    0.811210      811210         1 avformat_open_input
  0.75    0.085290         584       146 av_freep
  0.49    0.055150         434       127 strlen
  0.29    0.033008         660        50 av_log
  0.22    0.025090         464        54 strcmp
  0.20    0.022836       22836         1 avformat_close_input
  0.16    0.017788         635        28 av_dict_free
  0.15    0.016819         646        26 av_dict_get
  0.15    0.016753         440        38 strchr
  0.13    0.014536         581        25 memset
...
------ ----------- ----------- --------- --------------------
100.00   11.318773                4762 total
```



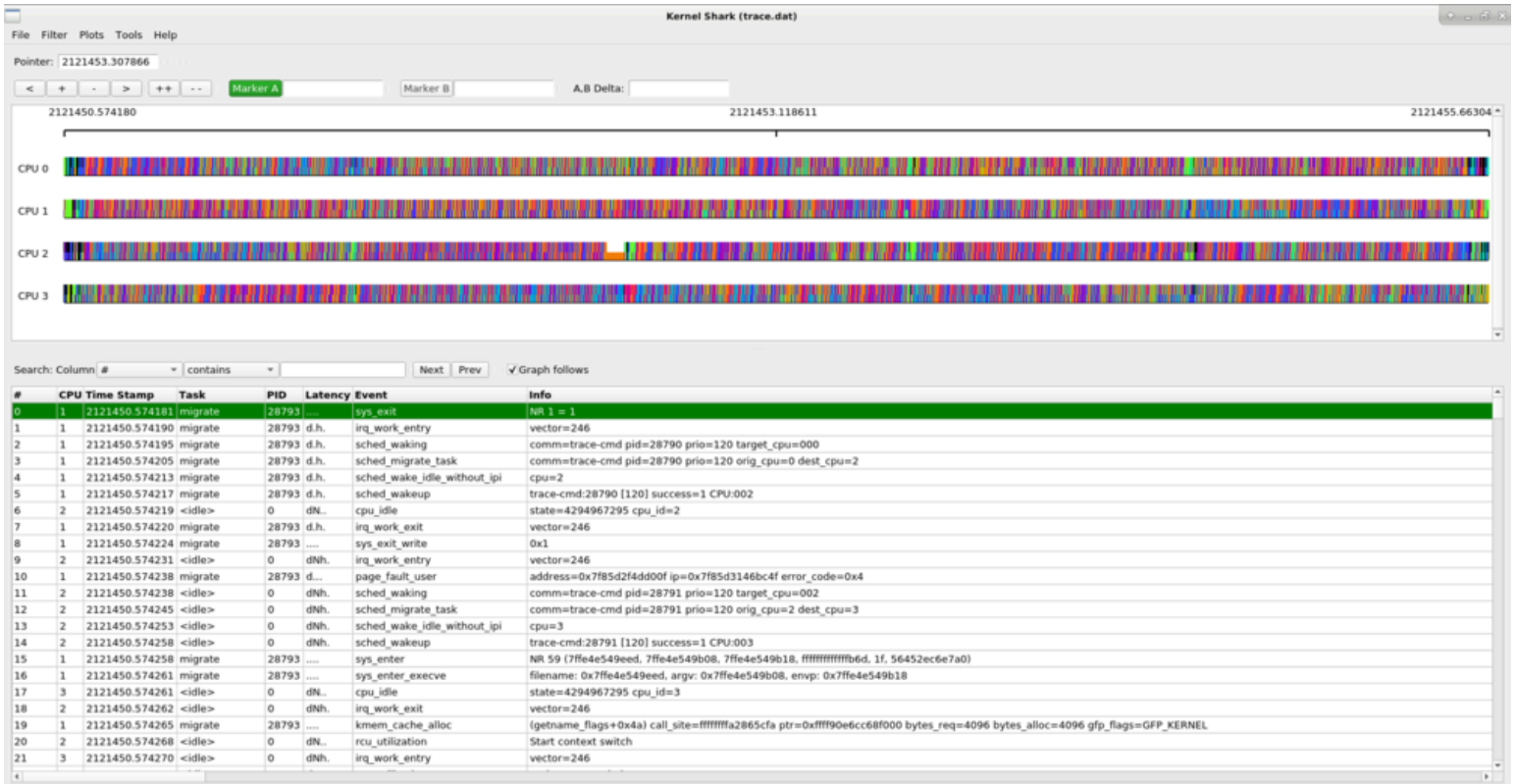
# ftrace

- In-kernel *tracing* functionality
- Can trace
  - Well-defined trace locations in the kernel, called *tracepoints*, identifying important events in the kernel: scheduling, interrupts, etc.
  - Arbitrary functions in the kernel
  - Arbitrary functions in user-space applications
- Low-overhead and optimized tracing
- Accessible using the dedicated *tracefs* filesystem
- `trace-cmd` is a higher-level CLI tool to use *ftrace*
- Can be used to understand overall system activity (what is my system doing?) as well as narrow down specific performance issues
- https://www.kernel.org/doc/Documentation/trace/ftrace.txt
- https://www.trace-cmd.org/



# kernelshark

- Visualization tool for *ftrace* traces
- https://kernelshark.org/





# perf

- *instrument CPU performance counters, tracepoints, kprobes, and uprobes*
- Directly included in the Linux kernel source code: tools/perf
- Began as a tool for using the performance counters in Linux, and has had various enhancements to add tracing capabilities
- Supports a list of measurable events: hardware events (cycle count, L1 cache hits/miss, page faults), software events (tracepoints)
- https://perf.wiki.kernel.org



# perf examples

- List all currently known events
  `perf list`
- List scheduler tracepoints
  'perf list `sched:*'`
- CPU counter statistics for the specified command
  `perf stat <command>`
- CPU counter statistics for the entire system, for 5 seconds
  `perf stat -a sleep 5`
- Profiling: sample on-CPU functions for the specified command, at 99 Hertz
  `perf record -F 99 <command>`
- Tracing: trace all context-switches via sched tracepoint, until Ctrl-C
  `perf record -e sched:sched_switch -a`
- Many more at https://www.brendangregg.com/perf.html



# perf GUI: hotspot

- Hotspot - the Linux perf GUI for performance analysis
- The main feature of hotspot is visualizing a `perf.data` file graphically
- github.com/KDAB/hotspot





# gprof

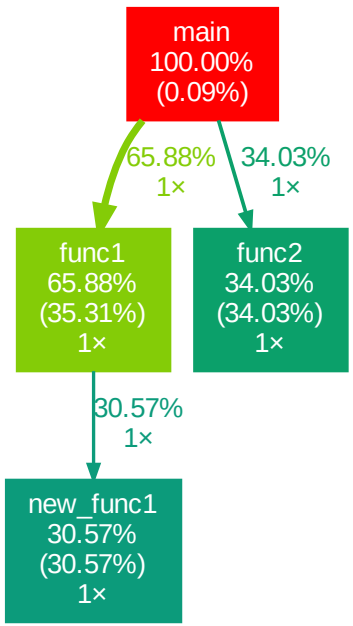
- Application-level profiler
- Part of *binutils*
- Requires passing gcc `-pg` option at build/link time
- Run your program normally, it automatically generates a `gmon.out` file when exiting
- Use the `gprof` tool on `gmon.out` to extract profiling data
- http://sourceware.org/binutils/docs/gprof/



# gprof example

```
$ ./test-gprof
$ gprof test-gprof gmon.out
Flat profile:

Each sample counts as 0.01 seconds.
  %   cumulative   self              self     total
 time   seconds   seconds    calls   s/call   s/call  name
 35.31      7.46     7.46        1     7.46    13.92  func1
 34.03     14.65     7.19        1     7.19     7.19  func2
 30.57     21.11     6.46        1     6.46     6.46  new_func1
  0.09     21.13     0.02                             main
[...]
```




Generated with gprof2dot





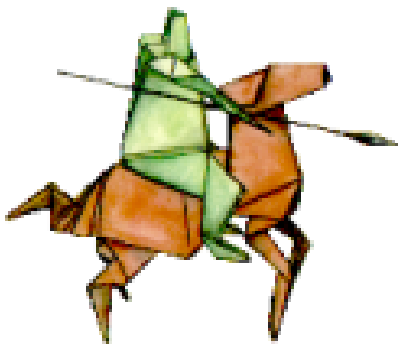
# Memory debugging



# Valgrind

https://valgrind.org/

- *instrumentation framework for building dynamic analysis tools*
  - detect many memory management and threading bugs
  - profile programs
- Supported architectures: x86, x86-64, ARMv7, ARMv8, mips32, s390, ppc32 and ppc64
- Very popular tool especially for debugging memory issues
- Runs your program on a synthetic CPU → significant performance impact (100 x slower on SAMA5D3!), but very detailed instrumentation
- Runs on the target. Easy to build with Yocto Project or Buildroot.





# Valgrind tools

- *Memcheck*: detects memory-management problems
- *Cachegrind*: cache profiler, detailed simulation of the I1, D1 and L2 caches in your CPU and so can accurately pinpoint the sources of cache misses in your code
- *Callgrind*: extension to Cachegrind, provides extra information about call graphs
- *Massif*: performs detailed heap profiling by taking regular snapshots of a program's heap
- *Helgrind*: thread debugger which finds data races in multithreaded programs. Looks for memory locations accessed by multiple threads without locking.
- More at https://valgrind.org/info/tools.html



# Valgrind examples

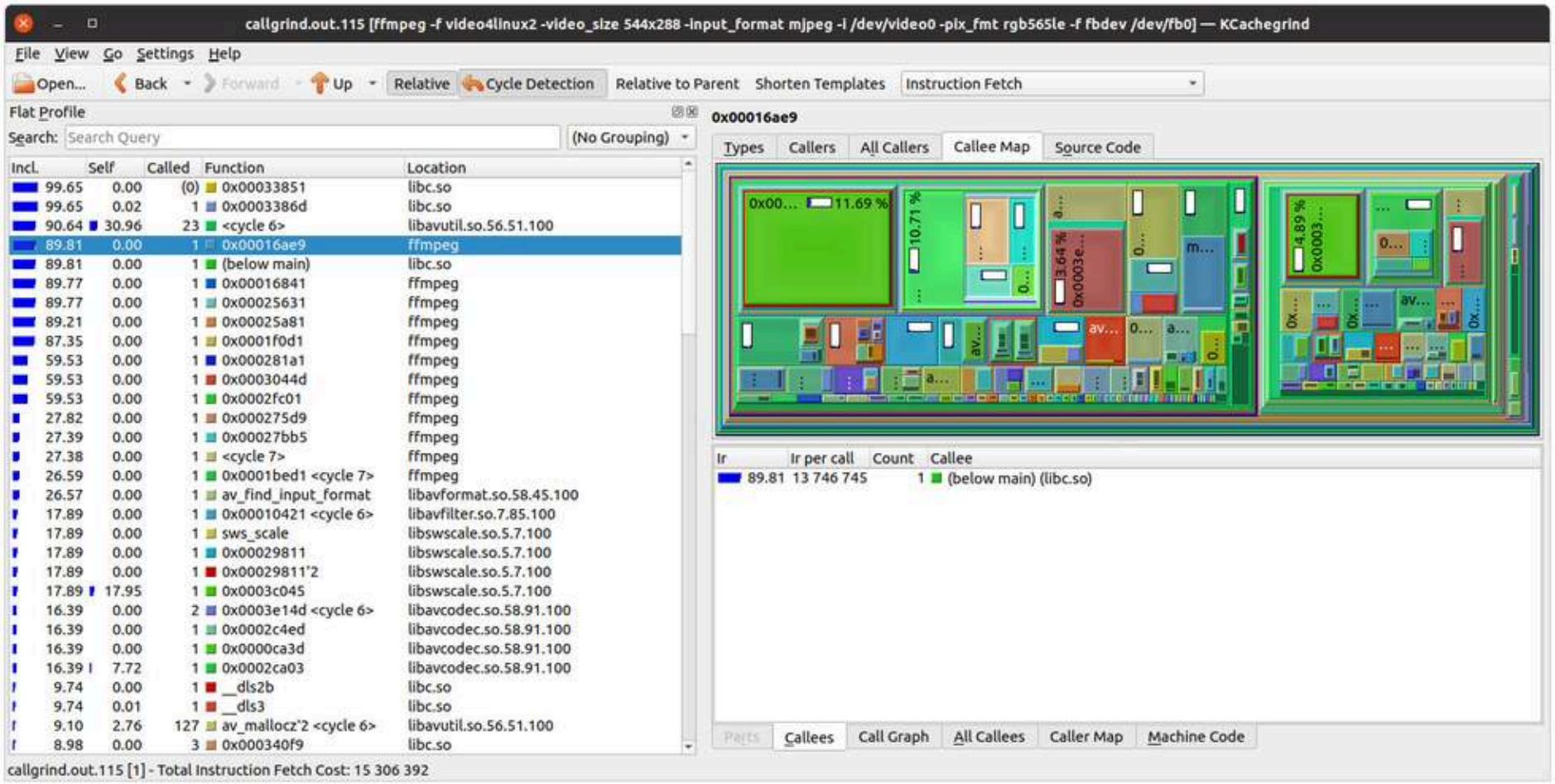
- *Memcheck*

```
$ valgrind --leak-check=yes <program>
  ==19182== Invalid write of size 4
  ==19182==    at 0x804838F: f (example.c:6)
  ==19182==    by 0x80483AB: main (example.c:11)
  ==19182==  Address 0x1BA45050 is 0 bytes after a block of size 40 alloc'd
  ==19182==    at 0x1B8FF5CD: malloc (vg_replace_malloc.c:130)
  ==19182==    by 0x8048385: f (example.c:5)
  ==19182==    by 0x80483AB: main (example.c:11)
```

- *Callgrind*

```
$ valgrind --tool=callgrind --dump-instr=yes --simulate-cache=yes --collect-jumps=yes <program>
$ ls callgrind.out.*
callgrind.out.1234
$ callgrind_annotate callgrind.out.1234
```

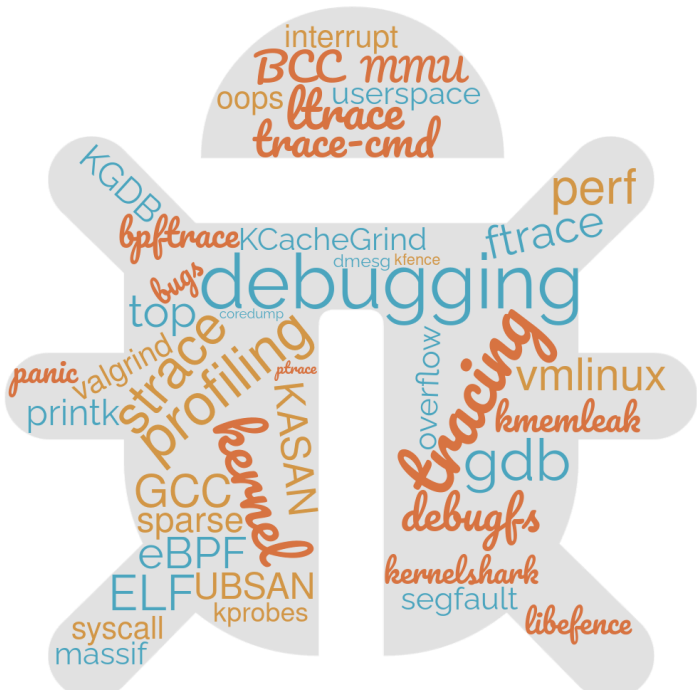
# Kcachegrind - Visualizing Valgrind profiling data



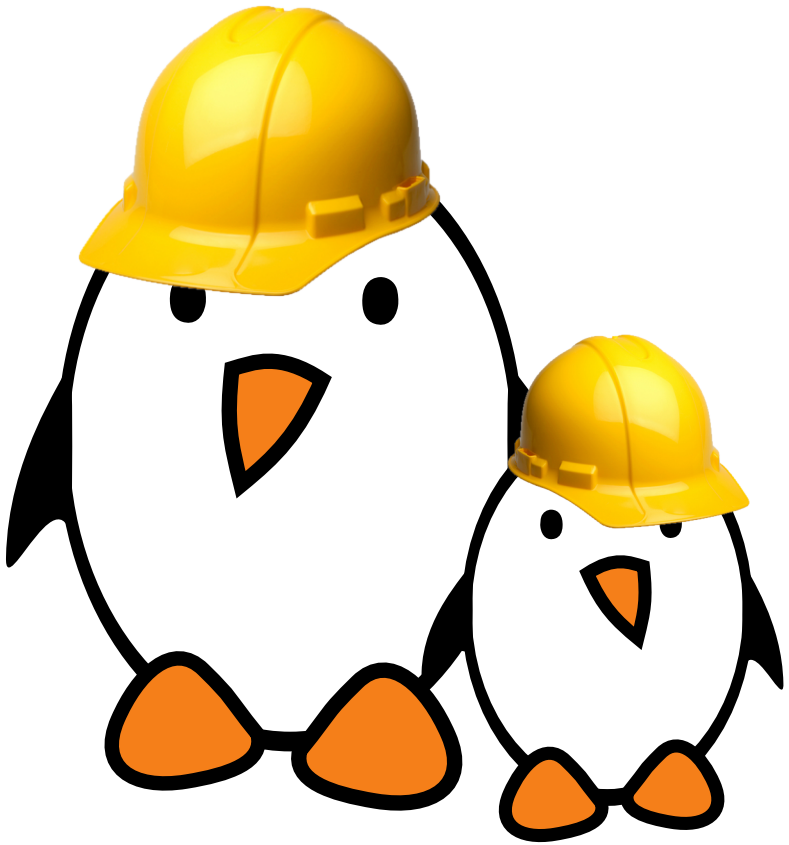

https://github.com/KDE/kcachegrind

# Debugging resources

- Brendan Gregg Systems performance book
- Brendan Gregg Linux Performance page
- Bootlin's "Linux debugging, profiling, tracing and performance analysis" training course and free training materials (250 pages): https://bootlin.com/training/debugging/.



# Practical lab - Application development and debugging



- Creating an application that uses an I2C-connected joystick to control an audio player.
- Setting up an IDE to develop and remotely debug an application.
- Using *strace*, *ltrace*, *gdbserver* and *perf* to debug/investigate buggy applications on the embedded board.